

■ 3.8 Numerical Operations on Data

■ 3.8.1 Curve Fitting

Built into *Mathematica* are various facilities for finding least-squares fits to data. The basic idea of the fits is to take a list of functions that you specify, and try to find a linear combination of them which approximates your data as well as possible. The goodness of fit is measured by the quantity $\chi^2 = \sum_i |F_i - f_i|^2$, where F_i is the value of your i^{th} data point, and f_i is the value obtained from the fit. The best fit is the one which minimizes χ^2 .

<code>Fit[data, funs, vars]</code>	fit a list of data points using the functions <i>funs</i> of variables <i>vars</i>
------------------------------------	--

The basic form of the `Fit` function.

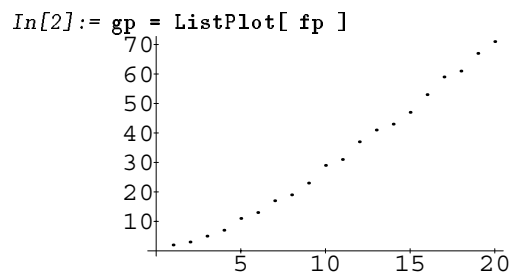
<code>Fit[{f₁, f₂, ...}, {1, x}, x]</code>	linear fit
<code>Fit[{f₁, f₂, ...}, {1, x, x^2}, x]</code>	quadratic fit
<code>Fit[data, Table[x^i, {i, 0, n}], x]</code>	n^{th} degree polynomial fit
<code>Exp[Fit[Log[data], {1, x}, x]]</code>	fit to e^{a+bx}

Some fits to lists of data.

Here is a table of the first 20 primes.

```
In[1]:= fp = Table[Prime[x], {x, 20}]
Out[1]= {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
         43, 47, 53, 59, 61, 67, 71}
```

Here is a plot of this "data".



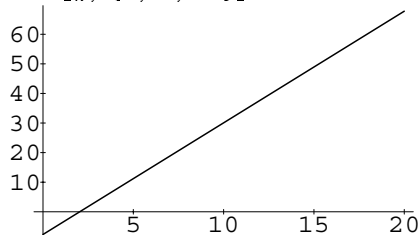
Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This gives a linear fit to the list of primes.
The result is the best linear combination of
the functions 1 and x .

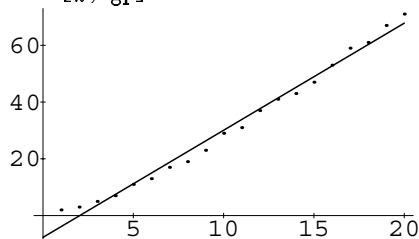
Here is a plot of the fit.

```
In[3]:= Fit[fp, {1, x}, x]
Out[3]= -7.67368 + 3.77368 x
In[4]:= Plot[%, {x, 0, 20}]
```



Here is the fit superimposed on the original
data.

```
In[5]:= Show[%, gp]
```

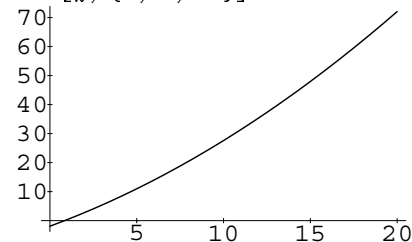


This gives a quadratic fit to the data.

```
In[6]:= Fit[fp, {1, x, x^2}, x]
Out[6]= -1.92368 + 2.2055 x + 0.0746753 x^2
```

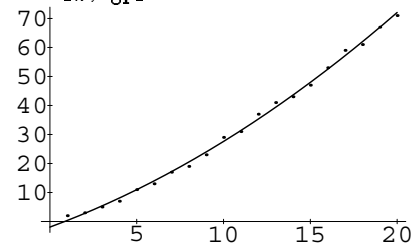
Here is a plot of the quadratic fit.

```
In[7]:= Plot[%, {x, 0, 20}]
```



This shows the fit superimposed on the
original data. The quadratic fit is better
than the linear one.

```
In[8]:= Show[%, gp]
```



Polynomial fits are the most common kind to use. However, if you have a reason to believe that your data follows some other functional form, you can include the appropriate functions in the list you give to **Fit**.

This gives a table of the values of $1 + 2e^{-x/3}$ for x from 1 to 10 in steps of 1.

```
In[9]:= ft = Table[ N[1 + 2 Exp[-x/3]] , {x, 10}]
Out[9]= {2.43306, 2.02683, 1.73576, 1.52719, 1.37775,
1.27067, 1.19394, 1.13897, 1.09957, 1.07135}
```

This fit recovers the original functional form.

```
In[10]:= Fit[ ft, {1, Exp[-x/3]}, x ]
Out[10]= 1. +  $\frac{2.}{E^{\frac{x}{3}}}$ 
```

If you include other functions in the list, **Fit** determines that they occur with small coefficients.

```
In[11]:= Fit[ ft, {1, Sin[x], Exp[-x/3], Exp[-x]}, x ]
Out[11]= 1. -  $\frac{9.76996 \cdot 10^{-15}}{E^x} + \frac{2.}{E^{\frac{x}{3}}} + 0. \cdot \text{Sin}[x]$ 
```

You can use **Chop** to get rid of the terms with small coefficients.

```
In[12]:= Chop[%]
Out[12]= 1. +  $\frac{2.}{E^{\frac{x}{3}}}$ 
```

There are several common reasons for doing fits.

If you have a particular model for some data, you can do a fit to try and determine the parameters of the model.

Another common use of fits is in finding approximate formulae to describe a particular set of data. You can use the form you get from a **Fit** as a summary of your actual data.

If you have only a few data points, then by including enough functions, you can get a fit which *exactly* reproduces the data you have. You can then interpolate or extrapolate from your data just by evaluating the fit at other points. If you use polynomial fits, you will find that you can always reproduce n data points with a polynomial of degree $n - 1$.

Here is a list of the first five factorials.

```
In[13]:= Table[x!, {x, 5}]
Out[13]= {1, 2, 6, 24, 120}
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This gives a fourth degree polynomial fit to the data.

```
In[14]:= fti = Fit[%, Table[x^i, {i, 0, 4}], x]
Out[14]= 45. - 93.75 x + 67.7917 x^2 - 20.25 x^3 +
          2.20833 x^4
```

Evaluating the fit for integer x between 1 and 5 reproduces your exact data.

```
In[15]:= fti /. x->3
Out[15]= 6.
```

Here is the interpolated value at $x = 2.5$.

```
In[16]:= fti /. x->2.5
Out[16]= 4.17969
```

This gives an extrapolation of your data, based on your fit.

```
In[17]:= fti /. x->8
Out[17]= 2311.
```

Like many extrapolations, it does not give a terribly accurate result.

```
In[18]:= 8!
Out[18]= 40320
```

In all the examples of **Fit** so far, the data points you give are assumed to correspond to the results of evaluating a function of one variable when the variable successively takes on values 1, 2, You can also specify data that depends on several variables, each given an arbitrary sequence of values, not necessarily arranged in any kind of regular array.

$\{f_1, f_2, \dots\}$ data points obtained when a single coordinate takes on values 1, 2,

$\{\{x_1, y_1, \dots, f_1\}, \{x_2, y_2, \dots, f_2\}, \dots\}$
data points obtained with values $x_i, y_i, \dots$ of a sequence of coordinates

Ways of specifying data in Fit.

This gives a table of the values of x, y and $1 + 2x + 3x - xy$. You need to use **Flatten** to get it in the right form for **Fit**.

```
In[19]:= Flatten[ Table[ {x, y, 1 + 5x - x y},
                        {x, 0, 1, .4}, {y, 0, 1, .4} ], 1]
Out[19]= {{0, 0, 1}, {0, 0.4, 1}, {0, 0.8, 1},
          {0.4, 0, 3.}, {0.4, 0.4, 2.84}, {0.4, 0.8, 2.68},
          {0.8, 0, 5.}, {0.8, 0.4, 4.68}, {0.8, 0.8, 4.36}}
```

This produces a fit to a function of two variables.

```
In[20]:= Fit[ %, {1, x, y, x y}, {x, y} ]
Out[20]= 1. + 5. x + 1.55431 10-15 y - 1. x y
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

`Fit` takes the list of functions you give, and finds the best fit to your data, according to the least squares criterion, using these functions. There is absolutely no guarantee that the fit you get will in fact accurately reproduce your data. To find out whether it can, you should use statistical testing functions, such as those in the statistics section of the *Mathematica* library.